



CLAUDE CODE: FROM FIRST INSTALL TO LIFE AFTER PRODUCTION

Skills: Free Until You Use Them

Claude Code deep dive, episode 6

The same hundred page document, at two completely different prices.

A skill is a folder with a markdown file in it, and the folder name becomes the command you type. That is the whole feature.

What makes it the highest leverage thing in the tool is the cost model: only the description loads at session start, and the body loads when the skill is used.

Claude Code changes with every release. Every figure, default and feature name on this sheet was checked on the date below and is stated as of that date. Check the current documentation before you rely on any of it.

In scope

- What a skill is, and the file that defines it
- Why the folder name matters
- The four locations, and what each one reaches
- On-demand loading, which is the whole argument
- The frontmatter worth knowing
- context: fork, and whose window pays
- Dynamic context injection
- That custom commands were merged into skills

Out of scope

- MCP servers, which are episode 7
- Subagent definition files, which are episode 8
- Plugin authoring and distribution
- Any specific community skill as a recommendation

Make one when you keep pasting the same instructions, or when a section of CLAUDE.md has grown into a procedure rather than a fact.

01 The four locations, and what each reaches

<code>~/.claude/skills/</code>	every project on this machine
<code>.claude/skills/</code>	anyone who clones the repository
<code><subdir>/.claude/skills/</code>	sessions at or below that path
managed settings	everyone in the organisation
The directory name	becomes the command
The description	decides when Claude reaches for it

These are interview assumptions chosen to make the behaviour concrete. They are not published figures from any provider and no claim is made about any particular deployment.



03 Smaller than people expect

Create a file called SKILL.md with instructions in it, and Claude adds it to its toolkit. It uses the skill when relevant, or you invoke it directly by name with a slash.

There is no framework, no registration step, and nothing to compile.

If you have been putting this off because it sounded like a project, it is a file.

Skills extend what Claude can do. Create a SKILL.md file with instructions, and Claude adds it to its toolkit. Claude uses skills when relevant, or you can invoke one directly with /skill-name.

Claude Code docs, Skills, checked September 2026

The directory name becomes the command name you type.

Claude Code docs, Skills, checked September 2026

04 The argument for the whole feature, in one sentence

At the start of a session Claude sees only the DESCRIPTIONS. The body of a skill loads only when the skill is actually used.

So your hundred page style guide in CLAUDE.md occupies your context window all day whether or not today's work touches that kind of code. The same guide as a skill costs you a one line description until the ten minutes you need it.

Same file, same words, two completely different prices.

Unlike CLAUDE.md content, a skill's body loads only when it is used, so long reference material costs almost nothing until you need it.

Claude Code docs, Skills, checked September 2026

Create a skill when you keep pasting the same instructions, checklist or multi-step procedure into chat, or when a section of CLAUDE.md has grown into a procedure rather than a fact.

Claude Code docs, Skills, checked September 2026



05 The location is the scope

Your home folder makes it available in every project on this machine. The project folder makes it available to anyone who clones the repository.

That second one is the move that turns a personal habit into a team standard, and it costs nothing extra.

A nested location loads only for sessions started at or below that path, which is how a monorepo gives different teams different tooling without collision. And an enterprise location deploys one to everybody.

Skills live in personal, project, nested and enterprise locations, and the location determines which sessions load them.

Claude Code docs, Skills, checked September 2026

06 The settings worth knowing

description is how Claude decides whether a skill is relevant, which makes it the most important line in the file and the one people write carelessly.

disable-model-invocation means only you can trigger it, never Claude, which keeps even the description out of the way until you ask. user-invocable set to false is the opposite.

allowed-tools pre-approves a specific set for that skill, so a read-only skill can be genuinely read-only. arguments gives it named positional arguments.

Frontmatter fields include name, description, disable-model-invocation, user-invocable, allowed-tools, context and arguments.

Claude Code docs, Skills, checked September 2026

A skill with context set to fork runs in an isolated subagent context with its own conversation history.

Claude Code docs, Skills, checked September 2026

07 The line of syntax that changes what a skill can be

Put an exclamation mark and a shell command in backticks, and Claude Code RUNS that command and substitutes the output before Claude ever sees the file.

So a skill is not a static instruction. It is a template that gathers its own evidence at the moment you invoke it.

A review skill that runs your diff and reviews what is actually there. A deploy checklist that reads the current branch. A triage skill that pulls the last fifty log lines. The instruction and the evidence arrive together.

The exclamation-backtick syntax uses dynamic context injection: Claude Code runs the command and replaces the line with its output before Claude sees the skill content.

Claude Code docs, Skills, checked September 2026

08 Something that quietly changed

Custom commands have been MERGED into skills. A file in the commands folder and a skill folder both create the same slash command, and both work.

Your existing command files keep working; nothing breaks.

What skills add on top is a folder for supporting files, frontmatter controlling whether you or Claude invokes them, and the ability for Claude to load them automatically when relevant. So a tutorial about custom commands is describing the smaller half of the same feature.

Custom commands have been merged into skills: a file at `.claude/commands/deploy.md` and a skill at `.claude/skills/deploy/SKILL.md` both create `/deploy` and work the same way.

Claude Code docs, Skills, checked September 2026

Bundled skills include `/debug`, `/code-review`, `/batch` and `/loop`.

Claude Code docs, Skills, checked September 2026

Should this be a skill?

1. Have you pasted these instructions more than twice?
2. Has a section of CLAUDE.md become a procedure rather than a fact?
3. Does the reference only matter for some tasks? Then it is a skill, not CLAUDE.md
4. Does the team need it, or only you? That decides the folder
5. Is the description precise enough for Claude to know when it applies?
6. Should Claude be able to invoke it, or only you?
7. Would it be better if it gathered its own evidence first?
8. Does it read many files? Consider context: fork

Everything else

Site

<https://code-with-sam-dev.github.io>

YouTube

<https://www.youtube.com/@CodewithSam-Dev>

LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>

X

<https://x.com/CodeWithSamDev>

TikTok

<https://www.tiktok.com/@codewithsamdev>

Instagram

<https://www.instagram.com/codewithsamdev>

Facebook

<https://www.facebook.com/codewithsamdev>

GitHub

<https://github.com/code-with-sam-dev>

☕ Buy me a coffee

<https://buymeacoffee.com/codewithsam>

Buy me sadza

<https://buymeacoffee.com/codewithsam>

Skills

<https://code.claude.com/docs/en/skills>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Claude and Claude Code are products of Anthropic. This is an independent, unofficial guide produced by Code with Sam. It is not affiliated with, endorsed by, or sponsored by Anthropic, and no Anthropic artwork is reproduced here. Product names appear only to identify the software being discussed. Other names are the property of their respective owners.

Technical claims verified 2026-09-13 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.